JARVIS Can Now Make PPTs Hands-Free! (EP 18) ■■

by **Pulse of AI** | Official Architecture & Production Release

■■ CRITICAL SYSTEM DEPENDENCY ADVISORY:
JARVIS is engineered as a cumulative, modular AI operating system. Core submodules introduced across prior episodes (including hardware_control, software_control, screen_vision, skill_synthesizer, llm_router, and app_automator) are mandatory runtime dependencies. Viewers MUST watch all previous videos (Episodes 1 to 17) and extract the exact source code from their respective pinned comments, or JARVIS will fail initialization and will not function.

1. Episode 18 Production Specs & Runtime Quickstart

Stage	Deployment Command / Environmental Target	Functional Operation
Dependencies	pip install python-pptx pyautogui google-genai pyaudio pywin32 pyperclip psutil winsdk python-dotenv rich keyboard requests	Installs the new PowerPoint COM/DOM engine and multimodal live WebSocket dependencies.
Environment	Setup.py & .env (GOOGLE_API_KEY)	Configures persistent secure keyring storage and initializes Gemini 3.1 Flash Live.
Execution	python agent.py	Launches HUD, dual terminals, audio streams, and arms the autonomous slide compiler.

2. Complete Series Architecture Catalog (Episodes 1 – 17)

Part	Direct YouTube Link	Part	Direct YouTube Link
Part 1	youtube.com/shorts/4fvNmeZ0E68	**Part 10**	youtube.com/shorts/q8v89plnk-Y
Part 2	youtube.com/shorts/s2sz62HNmbE	**Part 11**	youtube.com/shorts/FJvn6oonfk8
Part 3	youtube.com/shorts/qhaxKI-aocU	**Part 12**	youtube.com/shorts/nbP_oir2YKc
Part 4	youtube.com/shorts/WvV8ss0jn2o	**Part 13**	youtube.com/shorts/8vIIph3quxk
Part 5	youtube.com/shorts/7DCTFvycGe4	**Part 14**	youtube.com/shorts/nZkJVA_NCbc
Part 6	youtube.com/shorts/ek-iZBw-viY	**Part 15**	youtube.com/shorts/O9v1zEAa0X0
Part 7	youtube.com/shorts/6kbS4zYSN2Q	**Part 16**	youtube.com/shorts/BkD897ekPeY
Part 8	youtube.com/shorts/FWWldND5qQ4	**Part 17**	youtube.com/shorts/A1nFAerovdE
Part 9	youtube.com/shorts/9ATHesGlVdA	**Pulse Channel**	youtube.com/@PulseOfAIHQ

3. Complete Code: ppt_engine.py (Autonomous Presentation Compiler & Slideshow Launcher)

```
"""
ppt_engine.py
Autonomous PowerPoint Generation & Slide Show Launch Engine for JARVIS (Episode 18).
Generates structured dark-mode tech decks and launches them in presentation mode.
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import os
import subprocess
import threading
import time
import pyautogui
from pptx import Presentation
from pptx.util import Inches, Pt
from pptx.dml.color import RGBColor
from pptx.enum.text import PP_ALIGN

OUTPUT_DIR = os.path.join(os.path.expanduser("~"), "Desktop", "Jarvis_Presentations")
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Cyberpunk / Tech Presentation Palette
COLOR_BG = RGBColor(11, 15, 25)        # Dark Slate #0B0F19
COLOR_CYAN = RGBColor(0, 229, 255)     # Neon Cyan
COLOR_WHITE = RGBColor(241, 245, 249)  # Text White
COLOR_MUTED = RGBColor(148, 163, 184)  # Muted Secondary Gray

def apply_background(slide):
    background = slide.background
    fill = background.fill
    fill.solid()
    fill.fore_color.rgb = COLOR_BG

def build_presentation(topic: str, slide_count: int = 4) -> str:
    prs = Presentation()
    prs.slide_width = Inches(13.333)  # 16:9 Widescreen standard
    prs.slide_height = Inches(7.5)
    blank_layout = prs.slide_layouts[6]

    clean_topic = topic.strip().title() if topic.strip() else "Autonomous Intelligence"

    # --- Slide 1: Title Slide ---
    slide1 = prs.slides.add_slide(blank_layout)
    apply_background(slide1)

    title_box = slide1.shapes.add_textbox(Inches(1.5), Inches(2.2), Inches(10.33), Inches(2.0))
    tf = title_box.text_frame
    tf.word_wrap = True

    p = tf.paragraphs[0]
    p.text = clean_topic.upper()
    p.font.size = Pt(44)
    p.font.bold = True
    p.font.color.rgb = COLOR_CYAN
    p.alignment = PP_ALIGN.LEFT

    p2 = tf.add_paragraph()
    p2.text = "Autonomous Intelligence Briefing • Compiled by JARVIS"
    p2.font.size = Pt(20)
    p2.font.color.rgb = COLOR_MUTED
    p2.space_before = Pt(14)

    # --- Structured Content Blueprints ---
    slide_blueprints = [
        ("01 // EXECUTIVE SUMMARY", [
            f"Strategic transition driven by modern {clean_topic} paradigms.",
            "Elimination of manual execution bottlenecks via deterministic autonomous runtime agents.",
            "Sub-second edge orchestration mapped across multi-tier networks."
        ]),
        ("02 // CORE ARCHITECTURE", [
            f"Distributed data ingestion pipelines purpose-built for {clean_topic}.",
```

```
            "High-throughput model inference running parallel to deterministic desktop APIs.",
            "Proactive telemetry loops safeguarding system health and memory usage."
        ]),
        ("03 // KEY PERFORMANCE METRICS", [
            "Latency Target: Maintained strictly under 1.5 seconds end-to-end.",
            "Error Recovery: Autonomous fallback to secondary inference bridges.",
            "Task Precision: 99.4% task completion across local workspace tools."
        ]),
        ("04 // STRATEGIC HORIZON", [
            f"Full-scale adoption of {clean_topic} across production systems.",
            "Zero-friction interfaces connecting speech, vision, and desktop action.",
            "Continuous self-evolving capabilities and hot-reloaded toolchains."
        ])
    ]

    for title, bullets in slide_blueprints[:max(1, slide_count - 1)]:
        slide = prs.slides.add_slide(blank_layout)
        apply_background(slide)

        # Header Box
        header_box = slide.shapes.add_textbox(Inches(1.2), Inches(0.8), Inches(10.5), Inches(1.0))
        htf = header_box.text_frame
        hp = htf.paragraphs[0]
        hp.text = title
        hp.font.size = Pt(28)
        hp.font.bold = True
        hp.font.color.rgb = COLOR_CYAN

        # Content Box
        content_box = slide.shapes.add_textbox(Inches(1.2), Inches(2.2), Inches(10.5), Inches(4.5))
        ctf = content_box.text_frame
        ctf.word_wrap = True

        for b in bullets:
            bp = ctf.add_paragraph()

            bp.text = f"•  {b}"
            bp.font.size = Pt(18)
            bp.font.color.rgb = COLOR_WHITE
            bp.space_before = Pt(16)

    safe_name = "".join(c for c in clean_topic if c.isalnum() or c in (' ', '_')).rstrip()
    filepath = os.path.join(OUTPUT_DIR, f"{safe_name.replace(' ', '_')}_Deck.pptx")
    prs.save(filepath)
    return filepath

def _launch_worker(filepath: str):
    """Spawns PowerPoint in a detached thread and enters slideshow mode."""
    try:
        os.startfile(filepath)
        time.sleep(2.5)  # Wait for PowerPoint UI thread to load presentation
        pyautogui.press("f5")  # Fullscreen Slide Show
    except Exception as e:
        print(f"[PPT_ENGINE] Launch failed: {e}")

def create_and_launch_ppt(topic: str, slide_count: int = 4) -> str:
    """Entry point for JARVIS: compiles deck and dispatches slideshow."""
    filepath = build_presentation(topic, slide_count=int(slide_count))
    threading.Thread(target=_launch_worker, args=(filepath,), daemon=True).start()
    return f"Successfully generated a {slide_count}-slide presentation on '{topic}' and launched the slide show, sir."
```

4. Complete Code: agent.py (Gemini Live Engine with Presentation Tool & Cyberpunk HUD)

```
"""
agent.py
Real-Time Gemini Live Engine with Semi-Transparent Cyberpunk HUD Console (Episode 18)
Autonomous App Orchestration, PowerPoint Engine & Local GPU Vision
Built by Swayam Dhawale, Founder of Pulse of AI.
"""

import asyncio
import datetime
import os
import sys
import time
import math
import struct
import threading
import tkinter as tk
from tkinter import scrolledtext
import pyaudio
import winsound
import keyboard
import requests
from dotenv import load_dotenv
from google import genai
from google.genai import errors, types
import websockets
from rich.console import Console

load_dotenv()
console = Console()

# --- REMOTE KILL-SWITCH (Pulse of AI) ---
STATUS_URL = "https://gist.githubusercontent.com/PulseOfAIDev/a9951c6e858aacbabb604a0ed23f6ab7/raw/status.json"

def verify_app_access():
    """Checks remote status from GitHub Gist and displays cloud-configured message."""

    try:
        fresh_url = f"{STATUS_URL}?t={int(time.time())}"
        response = requests.get(fresh_url, headers={"Cache-Control": "no-cache"}, timeout=4).json()

        status = str(response.get("app_status", "free")).strip().lower()
        message = response.get(
            "message",
            "The free beta access period has concluded. To activate a subscription tier or upgrade your account, visit: https://
pulseofaidev.github.io"
        )

        if status != "free":
            console.print(f"\n[bold red]■ [ACCESS RESTRICTED][/bold red] {message}\n")
            sys.exit(0)
    except Exception:
        pass

# Run verification immediately on boot
verify_app_access()

# --- SECURE KEYRING INTEGRATION ---
from Setup import get_or_prompt_api_key

# Hardware & Telemetry
from hardware_control import (
    get_brightness,
    get_volume,
    set_brightness,
    set_volume,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    restore_master_volume,
)

# Software, Media Telemetry & Navigation Tools
```

```
from software_control import (
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    is_media_actively_playing,
    navigate_shorts,
    skip_video,
    seek_video,
    toggle_video_view,
)

# Local Zero-Latency Vision & Autonomous Grounding Tools
from screen_vision import (
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
)

# Self-Modifying Tri-Core Dynamic Skill Synthesizer
from skill_synthesizer import (
    evolve_skill,
    list_active_skills,
)

# Fast Cloud Reasoning Tool
from llm_router import (
    query_deepseek_reasoning,
)

# Episode 17: Autonomous App Composer & Typing Engine
from app_automator import (
    open_and_compose,

)

# Episode 18: Autonomous PowerPoint Generator & Presentation Engine
from ppt_engine import (
    create_and_launch_ppt,
)

# Dynamically initialize Gemini client using the secure keyring setup
client = genai.Client(api_key=get_or_prompt_api_key())

FORMAT = pyaudio.paInt16
CHANNELS = 1
RATE_IN = 16000
RATE_OUT = 24000
CHUNK = 512

p = pyaudio.PyAudio()

model_is_speaking = False
media_is_playing_state = False

# --- Semi-Transparent Cyberpunk Neural HUD with Mini-Mode ---
class JarvisHUD:
    def __init__(self, send_callback):
        self.send_callback = send_callback
        self.is_compact = False

        self.root = tk.Tk()
        self.root.title("■ JARVIS NEURAL CONSOLE")
        self.root.geometry("640x520")
        self.root.configure(bg="#0B0F19")

        self.root.attributes("-topmost", True)
        self.root.attributes("-alpha", 0.88)
```

```
        self.header_frame = tk.Frame(self.root, bg="#0B0F19")
        self.header_frame.pack(fill=tk.X, padx=12, pady=(6, 2))

        self.header_label = tk.Label(
            self.header_frame,
            text="JARVIS NEURAL CONSOLE",
            font=("Consolas", 11, "bold"),
            fg="#00E5FF",
            bg="#0B0F19"
        )
        self.header_label.pack(side=tk.LEFT)

        self.toggle_btn = tk.Button(
            self.header_frame,
            text="[–] COMPACT",
            font=("Consolas", 8, "bold"),
            bg="#1E293B",
            fg="#38BDF8",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=6,
            command=self.toggle_mode
        )
        self.toggle_btn.pack(side=tk.RIGHT)

        self.sub_header = tk.Label(
            self.root,
            text="Dual-Mode Voice & Keyboard Station • Pulse of AI",
            font=("Consolas", 8),
            fg="#70829B",
            bg="#0B0F19"
        )
        self.sub_header.pack(pady=(0, 4))

        self.chat_display = scrolledtext.ScrolledText(
            self.root,
            wrap=tk.WORD,
            font=("Consolas", 10),
            bg="#111827",
            fg="#E2E8F0",
            insertbackground="#00E5FF",
            relief=tk.FLAT,
            padx=10,
            pady=8
        )
        self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True)
        self.chat_display.config(state=tk.DISABLED)

        self.chat_display.tag_config("user_voice", foreground="#38BDF8", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("user_text", foreground="#A78BFA", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_header", foreground="#4ADE80", font=("Consolas", 10, "bold"))
        self.chat_display.tag_config("jarvis_stream", foreground="#86EFAC", font=("Consolas", 10))
        self.chat_display.tag_config("tool", foreground="#FBBF24", font=("Consolas", 9, "italic"))
        self.chat_display.tag_config("system", foreground="#94A3B8", font=("Consolas", 9))

        self.input_frame = tk.Frame(self.root, bg="#0B0F19")
        self.input_frame.pack(fill=tk.X, padx=12, pady=(4, 10))

        self.entry_box = tk.Entry(
            self.input_frame,
            font=("Consolas", 10),
            bg="#1E293B",
            fg="#F8FAFC",
            insertbackground="#00E5FF",
            relief=tk.FLAT
        )
        self.entry_box.pack(side=tk.LEFT, fill=tk.X, expand=True, ipady=6, padx=(0, 6))
        self.entry_box.bind("<Return>", self.on_send)
```

```
        self.send_btn = tk.Button(
            self.input_frame,
            text="TRANSMIT",
            font=("Consolas", 9, "bold"),
            bg="#0284C7",
            fg="#FFFFFF",
            activebackground="#0369A1",
            activeforeground="#FFFFFF",
            relief=tk.FLAT,
            padx=14,
            command=self.on_send
        )
        self.send_btn.pack(side=tk.RIGHT, ipady=4)

    def toggle_mode(self):
        if not self.is_compact:
            self.chat_display.pack_forget()
            self.sub_header.pack_forget()
            self.root.geometry("450x85")
            self.toggle_btn.config(text="[+] EXPAND")
            self.is_compact = True
        else:
            self.root.geometry("640x520")
            self.sub_header.pack(pady=(0, 4), before=self.input_frame)
            self.chat_display.pack(padx=12, pady=4, fill=tk.BOTH, expand=True, before=self.input_frame)
            self.toggle_btn.config(text="[—] COMPACT")
            self.is_compact = False

    def append_log(self, text: str, tag: str = "system"):
        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, text + "\n", tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def append_stream_chunk(self, token: str, tag: str = "jarvis_stream"):

        self.chat_display.config(state=tk.NORMAL)
        self.chat_display.insert(tk.END, token, tag)
        self.chat_display.see(tk.END)
        self.chat_display.config(state=tk.DISABLED)

    def on_send(self, event=None):
        msg = self.entry_box.get().strip()
        if msg:
            self.append_log(f"YOU (HUD): {msg}", "user_text")
            self.entry_box.delete(0, tk.END)
            self.send_callback(msg)

    def run(self):
        self.root.mainloop()

hud_instance = None

def start_gui(loop, text_queue):
    global hud_instance
    def on_user_submit(message: str):
        asyncio.run_coroutine_threadsafe(text_queue.put(message), loop)

    hud_instance = JarvisHUD(send_callback=on_user_submit)
    hud_instance.run()

def log_to_gui(text: str, tag: str = "system"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_log, text, tag)

def stream_token_to_gui(token: str, tag: str = "jarvis_stream"):
    if hud_instance:
        hud_instance.root.after(0, hud_instance.append_stream_chunk, token, tag)

def calculate_rms(data: bytes) -> float:
    count = len(data) // 2
```

```
    if count == 0:
        return 0.0
    shorts = struct.unpack(f"{count}h", data)
    sum_squares = sum(s * s for s in shorts)
    return math.sqrt(sum_squares / count)

def play_sfx_chime():
    try:
        winsound.Beep(1200, 60)
        winsound.Beep(1800, 80)
    except Exception:
        pass

def play_startup_chime():
    try:
        winsound.Beep(600, 70)
        winsound.Beep(900, 70)
        winsound.Beep(1400, 120)
    except Exception:
        pass

def create_document_and_write(app_name: str, topic: str, word_count: int = 150, doc_type: str = "essay") -> str:
    return open_and_compose(app_name, topic, word_count=int(word_count), mode=doc_type)

def generate_presentation(topic: str, slide_count: int = 4) -> str:
    return create_and_launch_ppt(topic, slide_count=int(slide_count))

tools = [
    open_website,
    open_application,
    close_application,
    purge_background_tasks,
    control_media,
    get_current_media_info,
    navigate_shorts,

    skip_video,
    seek_video,
    toggle_video_view,
    set_volume,
    get_volume,
    set_brightness,
    get_brightness,
    get_battery_status,
    get_system_vitals,
    get_system_temperatures,
    analyze_screen,
    find_ui_element,
    skip_youtube_ad,
    evolve_skill,
    list_active_skills,
    query_deepseek_reasoning,
    create_document_and_write,
    generate_presentation,
]

def get_time_greeting() -> str:
    current_hour = datetime.datetime.now().hour
    if 5 <= current_hour < 12:
        return "Good morning, sir."
    elif 12 <= current_hour < 17:
        return "Good afternoon, sir."
    else:
        return "Good evening, sir."

async def monitor_playback_state():
    global media_is_playing_state
    last_state = None
    while True:
        try:
            current_state = await asyncio.to_thread(is_media_actively_playing)
```

```
            if current_state != last_state:
                media_is_playing_state = current_state
                last_state = current_state
                if current_state:
                    console.print("\n[bold yellow]■■ [MEDIA PLAYING DETECTED] Push-to-Talk Armed: Hold [Right Ctrl] to speak.[/
bold yellow]")
                    log_to_gui("[MEDIA DETECTED] Push-to-Talk: Hold [Right Ctrl] or type below.", "system")
                else:
                    console.print("\n[bold green]■ [MEDIA PAUSED/IDLE] Hands-Free Mode Active. Speak freely.[/bold green]")
                    log_to_gui("[MEDIA IDLE] Hands-Free Voice Mode Active.", "system")
        except Exception:
            pass
        await asyncio.sleep(1.0)

async def listen_terminal_input(text_queue):
    loop = asyncio.get_running_loop()
    while True:
        try:
            user_input = await loop.run_in_executor(None, sys.stdin.readline)
            if user_input:
                clean_text = user_input.strip()
                if clean_text:
                    log_to_gui(f"YOU (Terminal): {clean_text}", "user_text")
                    await text_queue.put(clean_text)
        except Exception:
            await asyncio.sleep(0.1)

async def handle_unified_text_input(session, text_queue):
    while True:
        try:
            text = await text_queue.get()
            if text:
                console.print(f"\r[bold magenta]■■ [TRANSMITTED INPUT][/bold magenta] {text}")
                await session.send_realtime_input(text=text)
        except Exception:
            await asyncio.sleep(0.05)


async def send_audio(session):
    global model_is_speaking, media_is_playing_state
    mic_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_IN,
        input=True,
        frames_per_buffer=CHUNK,
    )
    console.print("[cyan]■ [LISTENING] Dynamic Adaptive Audio Pipeline Online...[/cyan]")

    silence_frame = b'\x00' * (CHUNK * 2)

    try:
        while True:
            data = await asyncio.to_thread(
                mic_stream.read, CHUNK, exception_on_overflow=False
            )

            if model_is_speaking:
                await session.send_realtime_input(
                    audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                )
                continue

            if media_is_playing_state:
                if keyboard.is_pressed("right ctrl"):
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                    )
```

```
            else:
                rms = calculate_rms(data)
                if rms > 450:
                    await session.send_realtime_input(
                        audio={"data": data, "mime_type": "audio/pcm;rate=16000"}
                    )
                else:
                    await session.send_realtime_input(
                        audio={"data": silence_frame, "mime_type": "audio/pcm;rate=16000"}
                    )

    except asyncio.CancelledError:
        raise
    finally:
        try:
            mic_stream.stop_stream()
            mic_stream.close()
        except OSError:
            pass

async def receive_audio(session, audio_queue):
    global model_is_speaking
    try:
        while True:
            async for response in session.receive():
                if response.tool_call is not None:
                    function_responses = []
                    for call in response.tool_call.function_calls:
                        await asyncio.to_thread(play_sfx_chime)

                        result = ""
                        if call.name == "open_website":
                            target_url = call.args.get("url", "https://google.com")
                            result = await asyncio.to_thread(open_website, target_url)
                        elif call.name == "open_application":

                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(open_application, str(target_app).strip())
                        elif call.name == "close_application":
                            target_app = (
                                call.args.get("app_name")
                                or call.args.get("app")
                                or call.args.get("name")
                                or (list(call.args.values())[0] if call.args else "")
                            )
                            result = await asyncio.to_thread(close_application, str(target_app).strip())
                        elif call.name == "purge_background_tasks":
                            result = await asyncio.to_thread(purge_background_tasks)
                        elif call.name == "control_media":
                            action = call.args.get("action", "play_pause")
                            result = await control_media(action)
                        elif call.name == "get_current_media_info":
                            result = await get_current_media_info()
                        elif call.name == "navigate_shorts":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(navigate_shorts, direction)
                        elif call.name == "skip_video":
                            direction = call.args.get("direction", "next")
                            result = await asyncio.to_thread(skip_video, direction)
                        elif call.name == "seek_video":
                            action = call.args.get("action", "forward")
                            result = await asyncio.to_thread(seek_video, action)
                        elif call.name == "toggle_video_view":
                            mode = call.args.get("mode", "fullscreen")
                            result = await asyncio.to_thread(toggle_video_view, mode)
                        elif call.name == "set_volume":
```

```
                            level = call.args.get("level", 70)
                            result = await asyncio.to_thread(set_volume, level)
                        elif call.name == "get_volume":
                            result = await asyncio.to_thread(get_volume)
                        elif call.name == "set_brightness":
                            level = call.args.get("level", 50)
                            result = await asyncio.to_thread(set_brightness, level)
                        elif call.name == "get_brightness":
                            result = await asyncio.to_thread(get_brightness)
                        elif call.name == "get_battery_status":
                            result = await asyncio.to_thread(get_battery_status)
                        elif call.name == "get_system_vitals":
                            result = await asyncio.to_thread(get_system_vitals)
                        elif call.name == "get_system_temperatures":
                            result = await asyncio.to_thread(get_system_temperatures)
                        elif call.name == "analyze_screen":
                            prompt = call.args.get("prompt", "Analyze what is visible on my screen.")
                            result = await analyze_screen(prompt)
                        elif call.name == "find_ui_element":
                            element_name = call.args.get("element_name", "")
                            result = await find_ui_element(element_name)
                        elif call.name == "skip_youtube_ad":
                            result = await skip_youtube_ad()
                        elif call.name == "evolve_skill":
                            skill_description = call.args.get("skill_description", "")
                            result = await asyncio.to_thread(evolve_skill, skill_description)
                        elif call.name == "list_active_skills":
                            result = await asyncio.to_thread(list_active_skills)
                        elif call.name == "query_deepseek_reasoning":
                            query = call.args.get("query", "")
                            result = await asyncio.to_thread(query_deepseek_reasoning, query)
                        elif call.name == "create_document_and_write":
                            target_app = call.args.get("app_name", "notepad")
                            target_topic = call.args.get("topic", "")
                            target_words = call.args.get("word_count", 150)

                            target_mode = call.args.get("doc_type", "essay")
                            result = await asyncio.to_thread(create_document_and_write, target_app, target_topic, target_words,
target_mode)
                        elif call.name == "generate_presentation":
                            target_topic = call.args.get("topic", "Autonomous AI")
                            target_slides = call.args.get("slide_count", 4)
                            result = await asyncio.to_thread(generate_presentation, target_topic, target_slides)

                        console.print(f"\n[bold yellow]■■ [TOOL EXECUTED: {call.name}][/bold yellow] [green]{result}[/green]")
                        log_to_gui(f"■■ [{call.name}] {result}", "tool")

                        function_responses.append(
                            types.FunctionResponse(
                                name=call.name,
                                id=call.id,
                                response={"result": result},
                            )
                        )

                    if function_responses:
                        await session.send_tool_response(function_responses=function_responses)

                server_content = response.server_content
                if server_content is not None:
                    if server_content.interrupted:
                        while not audio_queue.empty():
                            audio_queue.get_nowait()
                        model_is_speaking = False

                    if server_content.input_transcription:
                        user_spoken = server_content.input_transcription.text
                        if user_spoken and user_spoken.strip():
                            console.print(f"\r[bold white][USER (Voice)][/bold white] {user_spoken.strip()}")
                            log_to_gui(f"YOU (Voice): {user_spoken.strip()}", "user_voice")

                    model_turn = server_content.model_turn
```

```
                    if model_turn is not None:
                        if not model_is_speaking:
                            model_is_speaking = True
                            console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                            stream_token_to_gui("JARVIS: ", "jarvis_header")

                        for part in model_turn.parts:
                            if part.text:
                                sys.stdout.write(part.text)
                                sys.stdout.flush()
                                stream_token_to_gui(part.text, "jarvis_stream")

                            if part.inline_data and part.inline_data.data:
                                await audio_queue.put(part.inline_data.data)

                    if hasattr(server_content, "output_transcription") and server_content.output_transcription:
                        transcribed_token = server_content.output_transcription.text
                        if transcribed_token:
                            if not model_is_speaking:
                                model_is_speaking = True
                                console.print("[bold cyan]■ [JARVIS][/bold cyan] ", end="")
                                stream_token_to_gui("JARVIS: ", "jarvis_header")

                            sys.stdout.write(transcribed_token)
                            sys.stdout.flush()
                            stream_token_to_gui(transcribed_token, "jarvis_stream")

                    if server_content.turn_complete:
                        model_is_speaking = False
                        stream_token_to_gui("\n\n", "system")
                        console.print("\n[cyan]■ [LISTENING] Awaiting instructions...[/cyan]")

    except asyncio.CancelledError:
        raise
    finally:

        await audio_queue.put(None)

async def play_audio(audio_queue):
    speaker_stream = p.open(
        format=FORMAT,
        channels=CHANNELS,
        rate=RATE_OUT,
        output=True,
        frames_per_buffer=CHUNK,
    )
    try:
        while True:
            audio_data = await audio_queue.get()
            if audio_data is None:
                break
            await asyncio.to_thread(speaker_stream.write, audio_data)
    except asyncio.CancelledError:
        raise
    finally:
        try:
            speaker_stream.stop_stream()
            speaker_stream.close()
        except OSError:
            pass

async def run_session(config, model_id, greeting, text_queue):
    async with client.aio.live.connect(model=model_id, config=config) as session:
        console.print("[bold green]✔ Secure Neural Bridge Established with Gemini Live.[/bold green]")
        log_to_gui("✔ Neural Bridge Established with Gemini Live.", "system")

        restore_master_volume(70)
        await asyncio.to_thread(play_startup_chime)

        audio_queue = asyncio.Queue()
        monitor_task = asyncio.create_task(monitor_playback_state())
```

```
        send_task = asyncio.create_task(send_audio(session))
        term_task = asyncio.create_task(listen_terminal_input(text_queue))
        text_task = asyncio.create_task(handle_unified_text_input(session, text_queue))
        receive_task = asyncio.create_task(receive_audio(session, audio_queue))
        playback_task = asyncio.create_task(play_audio(audio_queue))

        startup_prompt = f"Say aloud: 'All protocols initialized. {greeting}'"
        await session.send_realtime_input(text=startup_prompt)

        try:
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, playback_task)
        finally:
            monitor_task.cancel()
            send_task.cancel()
            term_task.cancel()
            text_task.cancel()
            receive_task.cancel()
            playback_task.cancel()
            await asyncio.gather(monitor_task, send_task, term_task, text_task, receive_task, playback_task, return_exceptions=T
rue)

async def main():
    model_id = "gemini-3.1-flash-live-preview"
    greeting = get_time_greeting()

    config = types.LiveConnectConfig(
        response_modalities=["AUDIO"],
        tools=tools,
        realtime_input_config=types.RealtimeInputConfig(
            automatic_activity_detection=types.AutomaticActivityDetection(
                prefix_padding_ms=100,
                silence_duration_ms=300,
            )
        ),
        input_audio_transcription=types.AudioTranscriptionConfig(),
        output_audio_transcription=types.AudioTranscriptionConfig(),

        speech_config=types.SpeechConfig(
            voice_config=types.VoiceConfig(
                prebuilt_voice_config=types.PrebuiltVoiceConfig(
                    voice_name="Puck"
                )
            )
        ),
        system_instruction=types.Content(
            parts=[
                types.Part.from_text(
                    text=(
                        "You are JARVIS, an advanced artificial intelligence assistant created by "
                        "Swayam Dhawale, Founder of Pulse of AI. "
                        "Persona: Calm, highly sophisticated, articulate, and respectful, exactly like Tony Stark's JARVIS. "
                        "Always address the user as 'sir'. "
                        "CRITICAL LANGUAGE ADAPTATION RULE: "
                        "Match the exact language and dialect spoken or typed by the user. "
                        "If the user speaks or types in Hindi or Hinglish (e.g., 'PPT banao', 'essay likho', 'kya haal hai'), "
                        "you MUST reply completely in fluent, natural Hindi or polished Hinglish with 'sir'. "
                        "Never force English if the user initiated the interaction in Hindi. "
                        "Omni-Input Rule: Treat text input from the HUD console and PowerShell terminal with identical priority
to voice input. "
                        "Tool Execution Rules: "
                        "1. When opening any website or URL, call open_website and say: 'Yes sir, opening [site name]' (in Hindi
: 'Ji sir, [site name] khol raha hoon'). "
                        "2. When opening desktop apps, games, or tools, call open_application and say: 'Yes sir, opening [app na
me]' (in Hindi: 'Ji sir, [app name] shuru kar raha hoon'). "
                        "3. When asked to close, shut down, terminate, or kill ANY app, browser, window, or game, call close_app
lication with the app name. "
                        "4. When asked to kill hidden background tasks, purge background processes, clean orphan threads, or opt
imize RAM, call purge_background_tasks. "
                        "5. When asked what song, video, or track is playing, or what media is active, call get_current_media_in
fo. "
                        "6. When asked to pause, play, resume music or video, skip track, previous track, or mute/unmute audio,
call control_media with the action ('play_pause', 'next', 'previous', 'stop', 'mute'). "
                        "7. When asked to scroll shorts, go to next short, or next reel, call navigate_shorts with direction='ne
xt'. "
                        "8. When asked for next video, skip to next video in playlist, or next YouTube video, call skip_video wi
th direction='next'. "
                        "9. When asked to fast forward, skip 10 seconds ahead, or skip forward, call seek_video with action='for
ward'. "
                        "10. When asked to toggle full screen, enter full screen, or exit full screen, call toggle_video_view. "
                        "11. When setting volume, call set_volume. "
                        "12. When asked what the volume level is, call get_volume. "
                        "13. When setting brightness, call set_brightness. "
```

```
                        "14. When asked what the brightness is, call get_brightness. "
                        "15. When asked about battery, power, or charging state, call get_battery_status. "
                        "16. When asked for temperatures, heat, thermal readings, or how hot the rig is running, call get_system
_temperatures. "
                        "17. When asked for full diagnostics, performance vitals, system status, or complete CPU/GPU/RAM stats,
call get_system_vitals. "
                        "18. When asked to inspect the screen, read an error, examine the terminal, or explain what is visible o
n display, call analyze_screen. "
                        "19. When asked to locate or find a general UI icon, link, or window element, call find_ui_element. "
                        "20. MANDATORY AD SKIPPING RULE: Whenever the user says 'skip ad', 'skip this ad', 'skip video ad', 'cli
ck skip', or any variation of skipping/bypassing advertisements, you MUST IMMEDIATELY call skip_youtube_ad. "
                        "21. When asked to learn a new skill, teach yourself a tool, write new Python code into your core, or ev
olve a capability, call evolve_skill. "
                        "22. When asked what dynamic skills, new tools, or custom functions you have learned, call list_active_s
kills. "
                        "23. When asked to solve complex math, reason deeply, calculate probabilities, or when the user says 'as
k DeepSeek' or 'use R1', call query_deepseek_reasoning and state only the resulting answer directly to the user once, sir. "
                        "24. When the user asks to open Notepad, Word, or PowerPoint and write an essay or notes, but has NOT sp
ecified the length, DO NOT call create_document_and_write immediately. Ask aloud: 'How many words would you like the essay to be
, sir?' (in Hindi: 'Aap kitne shabdon ka essay chahte hain, sir?'). Once the user specifies the count, call create_document_and_
write with the app_name, topic, word_count, and doc_type. "
                        "25. MANDATORY PRESENTATION RULE: When the user asks to make, create, build, or present a PowerPoint pre
sentation, PPT, or slide deck on any topic (e.g., 'Make a PPT on AI', 'AI par presentation banao', 'PPT bana do'), call generate
_presentation with the topic and optional slide_count. "
                        "In Hindi, you MUST say aloud: 'PPT ban gayi hai sir, abhi khol raha hoon'. In English, say: 'The presen
tation is ready sir, launching the slide show now'. "
                        "General Rule: If the user asks a conversational question like 'can you hear me', simply respond politel
y with 'Yes sir, I hear you loud and clear'. Never execute media or control tools unless explicitly commanded to do so. "
                        "Respond in short, crisp sentences. Never speak markdown or bullet points. "
                        "If asked about your identity or creator, state clearly that you were built by Swayam Dhawale."
                    )
                )
            ]
        ),
    )

    console.print("[bold cyan]■■■
```